

Paso 11 - Validaciones | Ejercicios

Todo lo de esta hoja se agrega a tu clase `Validador`, y se prueba desde un `main`. **Copia tu `Validador.java` a la carpeta donde trabajes** (si lo perdiste, está en [referencia/Validador.java](#)) y compila siempre los dos archivos juntos: `javac Validador.java B11_1_LeerSiNo.java`.

Acuérdate de las dos familias:

- `es...` / `esta...` preguntan: reciben un dato, devuelven `true` o `false`, y no leen ni imprimen nada.
- `leer...` insisten: usan el `Scanner` y no sueltan al usuario hasta que el dato sirve.

► **Plantillas:** [plantillas/](#) — un `.java` por ejercicio, con el enunciado, la salida esperada y los `// TODO` donde va tu código. Si las usas, el archivo ya viene creado (`B11_1_LeerSiNo.java`) y **ese** nombre manda sobre el que diga el enunciado.

✓ **Solucionario:** [soluciones/](#) — el código de cada ejercicio, la salida real y por qué se resuelve así. Ábrelo **después** de intentarlo.

Cómo se lee cada ejercicio: el título dice qué hay que lograr; el bloque de abajo es la **salida esperada**, o sea lo que tiene que aparecer en tu pantalla (en los programas que preguntan cosas, lo que va después de la pregunta es lo que **escribe el usuario**); las citas como esta son pistas. Si no sabes por dónde partir, abre la plantilla: los `// TODO` numerados son los pasos, en orden. Compila y ejecuta después de cada uno.

B11.1 - `leerSiNo`

Qué practicas: construir un método `leer...` nuevo reutilizando otro que ya tienes, en vez de leer del `Scanner` de nuevo.

Agrega a `Validador` este método, al final de la familia de los `leer...`:

```
public static boolean leerSiNo(Scanner sc, String pregunta)
```

Devuelve `true` con `s` o `S`, `false` con `n` o `N`, e insiste con cualquier otra cosa.

```
Seguir? (s/n): tal vez
    Responde s o n.
Seguir? (s/n): S
```

Pista: `respuesta.equalsIgnoreCase("s")`. Escríbelo usando el `leerTexto` que ya tienes, no un `nextLine()` nuevo.

B11.2 - La clave segura

Qué practicas: recorrer un texto comprobando varias condiciones a la vez con variables `boolean`, y usar el resultado en un `while` de validación.

Agrega a `Validador`, esta vez en la familia de los que **preguntan**:

```
public static boolean esClaveSegura(String clave)
```

Una clave sirve si tiene **8 caracteres o más, al menos un dígito y al menos una mayúscula**.

```
esClaveSegura("duoc")           -> false
esClaveSegura("duocuc2026")     -> false
esClaveSegura("DuocUc2026")     -> true
```

Después úsala en un `while` que pida la clave hasta que sirva, y que diga **qué** le falta en cada intento.

Pistas: `clave.length()`, `clave.charAt(i)`, `Character.isDigit(c)`, `Character.isUpperCase(c)`. Necesitas dos variables `boolean` y un `for`.

B11.3 - El menú blindado

Qué practicas: blindar un programa que ya escribiste, reemplazando una lectura sin validar por una validada, sin tocar el resto del código.

Copia aquí tu menú del ejercicio B6.3 (el `do-while` completo) y reemplaza el `sc.nextInt()` que lee la opción por:

```
opcion = Validador.leerEntero(sc, "Opcion: ", 0, 3);
```

Acuérdate de copiar también tu `Validador.java` a esta carpeta.

Ahora escríbele `abc`, `-1` y `99`. El menú tiene que seguir vivo.

Pregunta: con `leerEntero` validando el rango, ¿sigue haciendo falta el `default` del `switch`? ¿Lo dejarías igual? Justifica.

B11.4 - El RUT

Qué practicas: usar `indexOf`, `substring` y `charAt` para comprobar el formato de un texto con reglas propias.

Otro de los que preguntan, en `Validador`:

```
public static boolean esRutValido(String rut)
```

Por ahora, un RUT sirve si: tiene **un** guion, antes del guion hay **solo dígitos** (al menos 7), y después del guion hay **un** carácter, que puede ser un dígito o una `k / K`.

```
esRutValido("12345678-5")    -> true
esRutValido("12345678-k")    -> true
esRutValido("12345678")      -> false
esRutValido("1234a678-5")    -> false
```

Pistas: `rut.indexOf('-')`, `rut.substring(0, guion)`, `rut.charAt(i)`, `Character.isDigit(c)`.

Extra (difícil): busca cómo se calcula el dígito verificador con el **módulo 11** y compruébalo de verdad. Vas a necesitar un `for` que recorra el cuerpo al revés multiplicando por 2, 3, 4, 5, 6, 7 y volviendo al 2.

El RUT vuelve en la demo 10, y ahí se valida de otra forma: lanzando una excepción en vez de devolver `false`. Guarda este ejercicio para compararlos.

B11.5 - Desafío: la ficha imposible de romper

Qué practicas: combinar todos los métodos de validación del paso en un solo programa, y comprobar con una lista de ataques que ninguno lo hace caer.

Junta todo en un solo programa que pida una ficha completa y que **no se pueda botar ni con letras, ni con vacíos, ni con datos fuera de rango**:

```
Nombre:           (no vacio)
RUT:              (formato valido)
Edad:             (15 a 99)
Correo DUOC:      (termina en @duocuc.cl)
Promedio:         (1.0 a 7.0)
Clave:            (segura)
Confirmar clave:  (igual a la anterior)
```

Al final muestra la ficha con `printf`, **sin** mostrar la clave.

La prueba de fuego. Cuando creas que está listo, córralo una vez y tírale **toda esta lista**, en orden, sin reiniciarlo:

#	Escribe esto	Tiene que pasar
1	Enter, sin escribir nada, en el nombre	lo vuelve a pedir
2	tres espacios en el nombre	lo vuelve a pedir
3	<code>12345678</code> en el RUT	lo vuelve a pedir (le falta el guion)
4	<code>abc</code> en la edad	lo vuelve a pedir, sin caerse
5	<code>14</code> y después <code>150</code> en la edad	los dos fuera de rango
6	<code>ana@gmail.com</code> en el correo	lo vuelve a pedir
7	<code>9.5</code> y después <code>0</code> en el promedio	los dos fuera de rango
8	<code>duoc</code> en la clave	insegura, la vuelve a pedir
9	una clave y una confirmación distintas	las vuelve a pedir

Si en alguno el programa se cae, o acepta el dato, **ahí tienes el arreglo**. El programa está listo cuando llega al final sin morirse ni una vez.

Cuando termines, sigue con el [paso 12](#).